Lucida

Installing the web app on a Linux server with an NVIDIA L40S (Ubuntu or Red Hat)

Step-by-step guide for package **Lucida-0.3.0-linux-x86_64.tar.gz**. It takes you from a server with the operating system already installed to examiners using Lucida in Chrome on their own PCs: the NVIDIA driver, getting and checking the package, installing the service, trusting its certificate, the first sign-in, the self-test, everyday operation and troubleshooting.

Server	x86_64, 64 GB RAM, NVIDIA L40S (48 GB), with Ubuntu Server 22.04 / 24.04 LTS or Red Hat Enterprise Linux 9 (Rocky Linux and AlmaLinux 9 work the same way as Red Hat)
One package	The same package installs on both families: it carries its own Python, CUDA libraries and FFmpeg. Only the NVIDIA driver installation differs, and this guide gives both
Examiner PCs	Google Chrome (or Edge) on Windows, macOS or Linux, on the same network
Time needed	About 30 minutes, plus a reboot if the NVIDIA driver is not installed yet
Internet	Only to install the NVIDIA driver (step 2). Lucida itself installs and runs fully offline

How to read this guide: grey boxes are commands to type on the server (or on a PC, where the box title says so). Type them exactly; lines starting with # are comments and need not be typed. Green boxes say what you should see. Orange boxes are things that matter for security or for evidence. Where Ubuntu and Red Hat differ, the box title says **Ubuntu** or **Red Hat**: follow only the one for your server.

Contents

Overview 3

Step 1. Log in to the server and find out which Linux it runs 4

Step 2. Install and check the NVIDIA driver 5

Step 3. Get the package onto the server 7

Step 4. Check the server 8

Step 5. Install Lucida 9

Step 6. Check that the service runs 11

Step 7. Trust the server's certificate on each examiner PC 11

Step 8. Open Lucida in Chrome and create the administrator 12

Step 9. Run and keep the acceptance self-test 13

Everyday operation 14

Troubleshooting 15

Quick reference 17

Overview

Lucida runs on the server as a system service called **lucida**. It serves its web interface over HTTPS on port **8443**. Examiners open `https://<server address>:8443` in Chrome on their own PCs; nothing is installed on those PCs except, once, the server's certificate authority, so Chrome trusts the connection. Evidence never leaves the server.

What the package contains

- A private Python 3.12 with every library Lucida uses. It does not touch the system's Python.
- ONNX Runtime for CUDA 12 with NVIDIA's CUDA 12 and cuDNN 9 libraries. The server therefore needs **only the NVIDIA driver**: no CUDA toolkit and no cuDNN installation.
- Static FFmpeg and FFprobe for video, the built web interface, the AI models (each pinned by SHA-256 and re-checked before every use), the documentation and the licences.
- `install.sh`, `check-server.sh` and, once installed, `/opt/lucida/uninstall.sh` and the `lucida` command.

Where things go

What	Where	Kept on upgrade / uninstall
Programs	`/opt/lucida`	Replaced / removed
Settings	`/etc/lucida/workbench.ini`	Kept / kept (removed with --purge)
Case data: evidence, derived files, audit log, accounts	`/var/lib/lucida`	Kept / kept (removed with --purge)
HTTPS certificate authority	`/var/lib/lucida/tls/workbench-ca.crt`	Kept / kept
Self-test logs	`/var/lib/lucida/logs`	Kept / kept
Service definition	`/etc/systemd/system/lucida.service`	Replaced / removed
Service log	`journalctl -u lucida`	System journal

The GPU and reproducibility

The learned models (super-resolution, face and object detection, plate reading) run on the L40S through CUDA. Lucida runs each model twice when it first loads it and uses the GPU only if both runs give the same bits; otherwise that model runs on the CPU. On the GPU it uses deterministic kernels and full 32-bit precision (no TF32), so a re-run after a restart gives the same result. Classical filters always run on the CPU, on every core, because they give the same bits on every machine. The report records where every step ran.

Before you start: checklist

- ☐ You can log in to the server as a user that may use `sudo` (on Red Hat: a member of *wheel*) (at the console, or with SSH from your PC).
- ☐ You know the server's IP address on the lab network, or will find it in step 1.

- [ ] The server can reach its package repositories (Ubuntu's archive, or Red Hat's plus NVIDIA's) for the NVIDIA driver (step 2), or the driver is already installed. If the server is fully offline, see *Offline driver installation* in step 2.
- [ ] You can download files from GitHub on some PC (step 3).
- [ ] Examiner PCs can reach the server on TCP port 8443 (no firewall between them blocks it).
- [ ] At least 20 GB free for the case data to start; 500 GB or more for real casework. A separate data disk is a good idea (see step 5).

Step 1. Log in to the server and find out which Linux it runs

Log in at the server's console, or from your PC with SSH. Windows 10 and 11 include `ssh`: open **PowerShell** or **Command Prompt** and type:

On your PC

```
ssh <your user>@<server address>
# for example:  ssh admin@10.20.30.40
```

1.1 Which operating system?

```
cat /etc/os-release
```

If you see	The server runs	Follow the boxes marked
NAME="Ubuntu", VERSION_ID="22.04" or "24.04"	Ubuntu Server LTS	**Ubuntu**
NAME="Red Hat Enterprise Linux", VERSION_ID="9.x"	Red Hat Enterprise Linux 9	**Red Hat**
NAME="Rocky Linux" or "AlmaLinux", VERSION_ID 9.x	A Red Hat 9 rebuild	**Red Hat** (differences noted)
VERSION_ID="8.x" (Red Hat, Rocky or Alma 8)	Too old for this package	Upgrade to 9 first (Red Hat's *leapp* tool), or reinstall with 9

Write down the family and the major version (for example *Red Hat 9*); step 2 depends on it. Everything from step 3 on is the same on both, except where a box says otherwise.

1.2 Processor, memory, disk and network

```
uname -m                # processor architecture
free -g                 # memory in GB
df -h / /var /opt       # free disk space
hostname -I             # the server's IP addresses
hostname -f             # the server's full name
```

> **You should see.** `x86_64`; about 62 in the *total* column of *Mem* (64 GB less what the firmware keeps); one or more addresses such as `10.20.30.40`. Write down the address examiners will use.

1.3 Bring the system up to date

If the server can reach the internet (or your lab's package mirror), update it first. This is normal maintenance, and the driver in step 2 must be built for the kernel that is running, so reboot after a kernel update before you continue.

Ubuntu

```
sudo apt update
sudo apt -y full-upgrade
sudo reboot
```

Red Hat (and Rocky / Alma)

```
sudo subscription-manager status     # Red Hat only: the server must be registered
sudo dnf -y upgrade
sudo reboot
```

Important. Make sure the clock is right: `timedatectl` should say *System clock synchronized: yes*. Audit-log entries and reports take their time from this clock. If it says no, point the server at your lab's time server: on Ubuntu edit `/etc/systemd/timesyncd.conf` (`NTP=<time server>`) and `sudo systemctl restart systemd-timesyncd`; on Red Hat edit `/etc/chrony.conf` (`server <time server> iburst`) and `sudo systemctl restart chronyd`.

Step 2. Install and check the NVIDIA driver

Lucida needs the NVIDIA driver, version **525 or newer** (550 or newer recommended for the L40S). It does **not** need the CUDA toolkit or cuDNN: those libraries are inside the package.

2.1 Is the card visible, and is a driver already installed?

```
lspci | grep -i nvidia
nvidia-smi
```

You should see. `lspci` lists a line with *NVIDIA Corporation AD102GL [L40S]*. If `nvidia-smi` prints a table naming *NVIDIA L40S* with about *46068MiB* of memory and a *Driver Version* of 550 or higher, the driver is ready: go to step 2.4.

If `lspci` shows nothing, the card is not seen by the server at all: check it is seated and powered, and that it is enabled in the BIOS (for a virtual machine: that the GPU is passed through to it). If `nvidia-smi` says *command not found*, install the driver (2.2). If it says *couldn't communicate with the NVIDIA driver*, see 2.3.

2.2 Ubuntu: install the driver from Ubuntu's archive

Ubuntu ships NVIDIA's data-centre drivers (the *-server* branches, which are the right ones for an L40S). Let Ubuntu list what fits the card, then install the newest *-server* branch it offers:

Ubuntu

```
sudo apt update
sudo apt install -y ubuntu-drivers-common pciutils
sudo ubuntu-drivers list --gpgpu
# the list shows names such as nvidia-driver-570-server and nvidia-driver-580-server.
# Install the highest number that ends in -server, for example:
sudo apt install -y nvidia-driver-570-server nvidia-utils-570-server
sudo reboot
```

Alternatively `sudo ubuntu-drivers install --gpgpu` picks the recommended branch on its own. Use one method, not both.

2.2 Red Hat: install the driver from NVIDIA's repository

Red Hat does not ship the NVIDIA driver; NVIDIA publishes it as a dnf repository. The driver is compiled for your kernel by DKMS, which comes from EPEL. For the L40S, NVIDIA recommends its *open* kernel modules. The commands are for version 9.

Red Hat 9 (and Rocky / Alma 9)

```
# 1. the CodeReady Builder repository (needed by EPEL)
sudo subscription-manager repos --enable codeready-builder-for-rhel-9-x86_64-rpms
#    Rocky / Alma 9 instead:  sudo dnf config-manager --set-enabled crb
# 2. EPEL, for DKMS
sudo dnf install -y \
  https://dl.fedoraproject.org/pub/epel/epel-release-latest-9.noarch.rpm
# 3. the build tools for the running kernel, and lspci
sudo dnf install -y kernel-devel-$(uname -r) kernel-headers-$(uname -r) pciutils
# 4. NVIDIA's repository
sudo dnf config-manager --add-repo \
  https://developer.download.nvidia.com/compute/cuda/repos/rhel9/x86_64/cuda-rhel9.repo
sudo dnf clean expire-cache
# 5. the driver (open kernel modules)
sudo dnf install -y nvidia-open                          # version 9
sudo reboot
```

If step 3 says no package matches `kernel-devel-...`, the running kernel is older than the newest available: run `sudo dnf -y upgrade`, reboot, and repeat from step 3. NVIDIA's proprietary modules (`sudo dnf install -y cuda-drivers`) work for Lucida too; choose one kind, not both.

Important. Secure Boot. Check with `mokutil --sb-state`. If it says *SecureBoot enabled*, the kernel only loads signed modules. **Ubuntu:** the driver installation asks you to choose a password. **Red Hat:** DKMS signs the module with its own key; register that key with `sudo mokutil --import /var/lib/dkms/mok.pub` and choose a password. On both, at the next reboot a blue *MOK management* screen appears on the server's console (use the BMC / iDRAC / iLO remote console): choose **Enroll MOK**, **Continue**, **Yes**, type that password, then **Reboot**. If you miss that screen the driver will not load; on Ubuntu run `sudo update-secureboot-policy --enroll-key`, on Red Hat the `mokutil --import` command again, and reboot.

Note. Do not install the CUDA toolkit (`nvidia-cuda-toolkit`, `cuda`, `cuda-toolkit-*`) or cuDNN for Lucida. They are not needed, and a second copy of the CUDA libraries can confuse other software on the server. Only the driver is needed.

2.3 After the reboot: check the driver

```
nvidia-smi
nvidia-smi --query-gpu=name,driver_version,memory.total,persistence_mode --format=csv
```

You should see. `NVIDIA L40S, 570.xx.xx, 46068 MiB, ...` (the numbers depend on the driver branch). The table's top line also shows *CUDA Version: 12.x* or higher; that is the highest CUDA the driver supports, not something you install.

If `nvidia-smi` still cannot talk to the driver:

- `lsmod | grep nouveau` shows a line: the open-source driver is still loaded. The NVIDIA package disables it; rebuild the boot image and reboot: Ubuntu `sudo update-initramfs -u`, Red Hat `sudo dracut -f`.

- `sudo dmesg | grep -i -E 'nvidia|secure'` mentions *Lockdown*, *key* or *signature*: Secure Boot rejected the module; enrol the key as described above.
- `dkms status` shows the module was not built for the running kernel: Ubuntu `sudo apt install --reinstall nvidia-dkms-570-server` (your branch number); Red Hat: install `kernel-devel-$(uname -r)`, then `sudo dkms autoinstall`; then reboot.

2.4 Turn on persistence mode

Persistence keeps the driver loaded while no program uses the card, so the first enhancement after a quiet period does not wait for the card to wake up:

```
sudo systemctl enable --now nvidia-persistenced
nvidia-smi --query-gpu=persistence_mode --format=csv,noheader
```

> **You should see.** `Enabled`.

Offline driver installation

If the server has no internet access, download the driver packages on a connected machine running the **same release** (same Ubuntu version, or same Red Hat major and minor version and kernel) and copy them across:

Ubuntu, offline

```
mkdir nvidia-debs && cd nvidia-debs
apt-get download $(apt-cache depends --recurse --no-recommends --no-suggests \
  --no-conflicts --no-breaks --no-replaces --no-enhances \
  nvidia-driver-570-server nvidia-utils-570-server | grep '^\w' | sort -u)
# copy the folder to the server, then on the server:
sudo apt install -y ./*.deb && sudo reboot
```

Red Hat, offline

```
# on the connected machine, after adding EPEL and NVIDIA's repository as above:
mkdir nvidia-rpms && cd nvidia-rpms
sudo dnf download --resolve --alldeps nvidia-open dkms kernel-devel-$(uname -r)
# copy the folder to the server, then on the server:
sudo dnf install -y ./*.rpm && sudo reboot
```

NVIDIA's own `.run` installer for the L40S (from nvidia.com, data-centre driver for Linux 64-bit) also works on both; it needs a compiler and the kernel headers on the server.

Step 3. Get the package onto the server

3.1 Download it

The package is built and tested by GitHub Actions, as the Windows installer is. On any PC with a browser:

- **From a release** (when one is published): open the repository's **Releases** page and download `Lucida-0.3.0-linux-x86_64.tar.gz` and `Lucida-0.3.0-linux-x86_64.tar.gz.sha256`. A release file can be at most 2 GB; if the package is larger it is published in parts (`Lucida-0.3.0-linux-x86_64.tar.gz.part00`, `.part01`, ...): download every part.
- **From the latest build**: open the repository's **Actions** tab, choose the **Linux server package** workflow, open the newest run on **main** with a green tick, and under **Artifacts** download **Lucida-Linux-Server**.

You must be signed in to GitHub. It downloads as a .zip that holds the .tar.gz and its .sha256; unzip it on your PC.

> **Note.** The package is about 2 GB, mostly NVIDIA's CUDA and cuDNN libraries. That is what lets the server work without a CUDA installation.

3.2 Copy it to the server

From the folder on your PC that holds the downloaded files (Windows PowerShell, macOS or Linux terminal; on Windows, WinSCP works too):

On your PC

```
scp Lucida-0.3.0-linux-x86_64.tar.gz* <your user>@<server address>:~/
```

If you downloaded parts, join them on the server first:

```
cat Lucida-0.3.0-linux-x86_64.tar.gz.part* > Lucida-0.3.0-linux-x86_64.tar.gz
```

3.3 Check it and unpack it

```
cd ~
sha256sum -c Lucida-0.3.0-linux-x86_64.tar.gz.sha256
tar -xzf Lucida-0.3.0-linux-x86_64.tar.gz
cd Lucida-0.3.0-linux-x86_64
ls
```

> **You should see.** `Lucida-0.3.0-linux-x86_64.tar.gz: OK`, then the folder holds `install.sh`, `check-server.sh`, `INSTALL.txt`, `PACKAGE.SHA256SUMS`, `linux-server-install.pdf` (this guide) and `payload/`. If the check says *FAILED*, the download is incomplete or damaged: download it again. Do not install a package that fails this check.

Your lab may record this SHA-256 with its validation records: `cat Lucida-0.3.0-linux-x86_64.tar.gz.sha256`.

Step 4. Check the server

`check-server.sh` looks at everything Lucida depends on and changes nothing. Run it from the unpacked folder:

```
./check-server.sh
```

Each line is **OK**, **WARN** (Lucida works, but not as well as it could) or **FAIL** (fix it first; the installer stops on a FAIL). On a correctly prepared server:

Example output

```
  OK    operating system: Red Hat Enterprise Linux 9.4 (Plow) (Red Hat family)
  OK    processor architecture: x86_64
  OK    C library: glibc 2.39
  OK    systemd is running
  OK    memory: 62 GB
  OK    processor: 32 threads (...)
  OK    free disk for case data (/var/lib): 812 GB
  OK    free disk for programs (/opt): 812 GB
  OK    graphics card on the PCI bus: 3D controller: NVIDIA Corporation AD102GL [L40S]
  OK    NVIDIA driver 570.xx.xx, supports CUDA 12.8; card NVIDIA L40S (46068 MiB)
  OK    the card is an L40S, as this package is tuned for
  OK    persistence mode is on
  OK    SELinux: Enforcing (the installer labels Lucida's folders; ...)
  OK    port 8443 is free
  OK    network addresses: 10.20.30.40
  OK    the clock is synchronised (audit-log times are right)
  All checks passed.
```

Message	What to do
driver ... is too old / no NVIDIA driver	Step 2.
nouveau driver is loaded	Step 2.3.
port 8443 is already used by ...	Another program uses the port. Install with another, e.g. `sudo ./install.sh --port 9443`.
free disk ... (at least 20 GB is needed)	Free space, or put the case data on another disk (step 5, `--data`).
lspci is not installed	Ubuntu `sudo apt install pciutils`; Red Hat `sudo dnf install pciutils`. Then run the check again.
memory: under 60 GB	Works; long video frame ranges are refused sooner (Lucida says how many frames fit).
the clock is not synchronised	Step 1, the orange box.
persistence mode is off	Step 2.4.

Step 5. Install Lucida

From the unpacked folder:

```
sudo ./install.sh
```

That is all most servers need. The installer:

- checks every file of the package against its SHA-256 and runs the server check again;
- creates the system account **lucida** (no login, no password) that the service runs as, and adds you to the group **lucida** so you can read logs and copy files for import;
- copies the programs to `/opt/lucida` and links the `lucida` command into `/usr/local/bin`;
- writes `/etc/lucida/workbench.ini`, sized for this server: with 64 GB and an L40S, 4 jobs at once on the CPU and 2 on the GPU (the L40S has room for two models side by side);
- creates `/var/lib/lucida` for the case data, readable only by **lucida** and its group;

- installs and starts the **lucida** service (it starts at every boot and restarts if it stops), opens port 8443 in the firewall if one is active (`ufw` on Ubuntu, `firewalld` on Red Hat, where it is on by default), restores the SELinux labels on Red Hat, and creates the HTTPS certificate authority for this server;
- runs the acceptance self-test and shows where the models run, then prints the addresses to open and the certificate's fingerprint.

It takes about five minutes, most of it the self-test. The end of the output looks like this:

Example output

```
== Graphics card
Graphics card:   NVIDIA L40S, 44 GB
ONNX Runtime:    1.26.0, providers CUDAExecutionProvider, CPUExecutionProvider
AI device rule:  auto (GPU via CUDAExecutionProvider when reproducible, else CPU)
Models (loaded, checked and run twice; the GPU is used only when both runs match):
  sr.fsrcnn_x2                    gpu (CUDAExecutionProvider)
  ...
RESULT: models run on the GPU

== Done
  Open Lucida in Chrome on an examiner PC at:
      https://10.20.30.40:8443
      https://lucida-srv.lab.local:8443

  First install the certificate authority on each examiner PC (once):
      /var/lib/lucida/tls/workbench-ca.crt  (copy: /home/admin/lucida-ca.crt)
      SHA-256 3A:7F:...:C2
  The first person to open Lucida creates the administrator account.
```

Note. If the last lines of the graphics-card part say *models run on the CPU*, Lucida works but slower; see *Troubleshooting*, *The models run on the CPU*.

Options

Option	Use it when
`--data /srv/lucida`	The case data should live on a separate, larger disk. Mount the disk first (and add it to /etc/fstab), then install with this option.
`--port 9443`	Port 8443 is taken, or your network allows another. --port 443 lets examiners type the address without a port.
`--hostname lucida.lab.local,10.0.0.5`	Examiners reach the server by a DNS alias or an address the server does not see on its own interfaces (NAT). The server's own name and addresses are always included.
`--no-sign-in`	Only for a closed test network: without sign-in, anyone who can reach the port can use Lucida.
`--local-only`	No network access at all: Lucida answers only on the server itself (http://127.0.0.1:8443).
`--no-firewall`	You manage the firewall yourself.
`--skip-selftest`	Skips the self-test (run lucida selftest later).
`-y`	Do not ask before continuing past warnings.

Note. Red Hat and SELinux. Leave SELinux in *Enforcing* mode. Lucida runs as an ordinary system service; the installer gives its folders the standard labels for programs (/opt), state (/var/lib) and settings (/etc). If you choose a data folder elsewhere with `--data`, the installer labels it as well.

Important. Sign-in is on by default, as it should be for a server on a shared network. Every action is recorded in the audit log under the signed-in person's name, and HTTPS keeps passwords off the network in clear.

Step 6. Check that the service runs

```
systemctl status lucida          # should say: active (running)
journalctl -u lucida -n 30       # the last lines of its log
lucida gpu                       # where the models run
```

You should see. `Active: active (running)`, and in the log lines like `Address: https://127.0.0.1:8443`, `Lab network: https://...` and `Uvicorn running on https://0.0.0.0:8443`. `lucida gpu` ends with *RESULT: models run on the GPU*.

While a model runs, `nvidia-smi` lists a *python3* process using the card. To watch it: `watch -n 1 nvidia-smi` (Ctrl+C to stop).

Step 7. Trust the server's certificate on each examiner PC

Lucida made its own small certificate authority (CA) for this server, with no internet. Chrome trusts Lucida once each PC trusts that CA. You do this once per PC; the CA is valid for ten years and survives upgrades, and the server certificate it signs is renewed automatically when the server's names or addresses change.

7.1 Copy the CA certificate to the PC

The installer put a copy in your home folder on the server. On the PC:

On the PC (PowerShell, or Terminal on macOS)

```
scp <your user>@<server address>:lucida-ca.crt .
```

Check that the file arrived unchanged: the two SHA-256 values must be the same.

On the server

```
sha256sum ~/lucida-ca.crt
```

On a Windows PC

```
certutil -hashfile lucida-ca.crt SHA256
```

On a Mac

```
shasum -a 256 lucida-ca.crt
```

Important. Install only a CA file whose SHA-256 matches. A CA in the trusted-root store can vouch for any web site to that PC, so it must be the one from your server. Lucida uses its CA only to sign this server's certificate, and its private key never leaves the server. Keep `/var/lib/lucida/tls/workbench-ca.key` private; it never needs to leave the server.

7.2 Windows (Chrome and Edge use the Windows certificate store)

With the mouse: double-click `lucida-ca.crt` > **Install Certificate...** > **Local Machine** > **Next** (approve the administrator prompt) > **Place all certificates in the following store** > **Browse...** > **Trusted Root Certification Authorities** > **OK** > **Next** > **Finish**.

Or in one command, in a PowerShell or Command Prompt opened with **Run as administrator**:

On a Windows PC, as administrator

```
certutil -addstore -f Root lucida-ca.crt
```

Then close every Chrome window (and check no Chrome icon remains near the clock) and open Chrome again. For many PCs, a domain administrator can push the CA with Group Policy: *Computer Configuration* > *Policies* > *Windows Settings* > *Security Settings* > *Public Key Policies* > *Trusted Root Certification Authorities* > *Import*.

7.3 macOS

On a Mac (Terminal)

```
sudo security add-trusted-cert -d -r trustRoot \
  -k /Library/Keychains/System.keychain lucida-ca.crt
```

Or double-click the file, add it to the **System** keychain, open it in Keychain Access, expand **Trust** and set **When using this certificate** to **Always Trust**. Restart Chrome.

7.4 Linux desktop

On a Linux PC

```
sudo apt install -y libnss3-tools
certutil -d sql:$HOME/.pki/nssdb -A -t "C,," -n "Lucida CA" -i lucida-ca.crt
```

Or in Chrome: **Settings > Privacy and security > Security > Manage certificates**, then import the file as a trusted authority for web sites. Restart Chrome.

Step 8. Open Lucida in Chrome and create the administrator

On the examiner PC, open Chrome and go to the address the installer printed, for example:

In Chrome's address bar

```
https://10.20.30.40:8443
```

> **You should see.** The Lucida sign-in screen, and a closed padlock (or the *tune* icon) at the left of the address bar with no warning. Clicking it shows *Connection is secure*.

If Chrome shows *Your connection is not private*, do not click through it: the CA is not trusted yet on this PC, or the address is not in the certificate. See *Troubleshooting*.

8.1 The first sign-in

The **first person** to open Lucida on a new installation creates the **administrator**: user name, full name and a password of at least 12 characters. Do this straight after installing, before you tell examiners the address.

8.2 Add the examiners

- Click your name at the top right > **Users...** and add each person with a role: **admin** (everything, user management), **examiner** (works on cases they are a member of), **reviewer** (reads member cases, records review decisions) or **viewer** (reads member cases only).
- Create a **second administrator** account and keep its password in your lab's safe. An administrator resets other people's passwords; with two, losing one password never locks you out.
- In each case, **Case > Who can open this case** lists its members. A case an examiner creates is theirs automatically.

Sessions end after 30 minutes without use and after 12 hours at most. Five wrong passwords lock an account for 15 minutes.

8.3 Tip: bookmark it, or install it as an app

Bookmark the address. In Chrome you can also choose the three-dot menu: **Cast, save and share > Install page as app** (older versions: **More tools > Create shortcut > Open as window**) so Lucida opens in its own window from the Start menu or the Dock.

Step 9. Run and keep the acceptance self-test

The installer already ran it. Run it again after every upgrade, driver update or hardware change, and keep its log with your lab's validation records:

```
lucida selftest
ls /var/lib/lucida/logs/
```

It re-hashes every installed file against the package's SHA256SUMS, checks FFmpeg, re-hashes every model, checks ONNX Runtime gives bit-identical output on this machine and names the device the models run on, then runs a small case end to end (ingest, render, reproduce, triage, measure, review, PDF report) in a temporary folder. Your case data is not touched.

> **You should see.** A list of PASS lines ending with *All checks passed.* and *Log saved to /var/lib/lucida/logs/selftest-<date>.txt*. The line *FSRCNN x2 reproducible on gpu (CUDAExecutionProvider)* confirms the L40S is used.

Other checks you can run at any time:

Command	What it checks
`lucida verify-install`	Every installed program file against the package's SHA-256 list.
`lucida verify`	Re-hashes every original in the evidence store and checks the audit log's hash chain.
`lucida gpu`	The card, the driver and where each model runs.
`lucida check`	The server check from step 4.
`lucida version`	Version, build date, commit and the hashes of the bundled components.

Everyday operation

Start, stop, restart, log

```
sudo systemctl stop lucida        # stop (examiners lose their connection)
sudo systemctl start lucida       # start
sudo systemctl restart lucida     # after changing settings
systemctl status lucida           # running or not
journalctl -u lucida -f           # follow the log (Ctrl+C to stop)
journalctl -u lucida --since today
```

A job that was running when Lucida stopped is marked failed and must be started again; jobs still waiting resume. Tell examiners before a restart.

Settings

Edit `sudo nano /etc/lucida/workbench.ini`, then `sudo systemctl restart lucida`. The most useful keys:

Key	Meaning
`port`	The HTTPS port (also open it in the firewall: Ubuntu sudo ufw allow <port>/tcp; Red Hat sudo firewall-cmd --permanent --add-port=<port>/tcp, then --reload).
`network`	lab = HTTPS for other PCs; local = this server only.
`auth`	accounts = sign-in with roles. Remove the line only on a closed test network.
`allowed_hosts`	Extra names or addresses examiners type, comma-separated (also added to the certificate).
`job_workers / gpu_jobs`	How many jobs run at once on the CPU / on the GPU (4 / 2 on this server).
`ai_device`	auto (GPU when reproducible), gpu or cpu.
`session_minutes`	Minutes without use before a session ends (default 30).
`tls_cert / tls_key`	Use your lab's own certificate instead of Lucida's CA (PEM files readable by group lucida).
`c2pa_trust_anchors`	PEM file of C2PA signers your lab trusts.
`data_dir / import_dir`	Case data and import folders. If you change them, run the installer again so the service may write there.

Bringing in evidence

Examiners upload files through Chrome; each file is hashed (SHA-256 and MD5) as it arrives and stored read-only. For large exports, copy them to the import folder on the server and ingest them from there; members of the group **lucida** may write to it:

```
cp -r /media/usb/DVR-export /var/lib/lucida/import/
lucida batch scan /var/lib/lucida/import/DVR-export
```

Backups

A backup is consistent while Lucida runs and lists every file's SHA-256. Keep backups on separate media and test a restore now and then:

```
# the backup folder must be writable by the lucida account
sudo install -d -o lucida -g lucida /mnt/backup/lucida
LUCIDA_BACKUP_PASSWORD='<a long password>' lucida backup \
  --output /mnt/backup/lucida/lucida-$(date +%F).backup --encrypt
# test a restore into an empty folder
# (it checks every file, re-hashes the originals and verifies the audit chain):
sudo install -d -o lucida -g lucida /mnt/backup/restore-test
lucida restore /mnt/backup/lucida/lucida-<date>.backup \
  --target /mnt/backup/restore-test
```

Note: the `lucida` command runs as the **lucida** account (it switches with sudo), so the folders you give it must be readable or writable by that account.

Upgrading

Get the new package (step 3), unpack it and run its installer. It stops the service, replaces the programs, keeps `/etc/lucida/workbench.ini`, the case data and the certificate authority (so examiner PCs need nothing new), starts the service and runs the self-test.

```
cd ~/Lucida-<new version>-linux-x86_64
sudo ./install.sh
```

A driver or kernel upgrade (`sudo apt upgrade` or `sudo dnf upgrade`, followed by a reboot) needs nothing from Lucida; run `lucida gpu` afterwards to confirm the GPU is still used.

Uninstalling

```
# remove the programs and the service; KEEP the case data and settings:
sudo /opt/lucida/uninstall.sh
# also delete the case data, the settings and the lucida account:
sudo /opt/lucida/uninstall.sh --purge
```

> **Important.** `--purge` deletes all evidence, derived files, the audit log and the accounts. It asks you to type DELETE and then to confirm. Back up first.

Troubleshooting

Start with these three commands; together they answer most questions:

```
systemctl status lucida
journalctl -u lucida -n 80 --no-pager
lucida check
```

What you see	Why	What to do
Chrome: *Your connection is not private*, NET::ERR_CERT_AUTHORITY_INVALID	This PC does not trust Lucida's CA yet.	Step 7 on this PC, into **Trusted Root Certification Authorities** of the **Local Machine**; then quit Chrome completely and reopen it.
Chrome: NET::ERR_CERT_COMMON_NAME_INVALID	The address you typed is not in the server certificate (a DNS alias, a NAT address).	Add it: in workbench.ini set `allowed_hosts = name1,10.0.0.5`, then `sudo systemctl restart lucida`. The certificate is re-issued; the CA stays the same. `lucida ca` lists the names it contains.

What you see	Why	What to do
The page says *Invalid host header* (400)	Lucida answers only names it knows, against DNS-rebinding attacks.	As above: add the name to `allowed_hosts` and restart.
Chrome: ERR_CONNECTION_REFUSED	Nothing listens on that port: the service is stopped, or a different port is set.	`systemctl status lucida`; `sudo ss -ltnp | grep 8443`; start it, or use the port in workbench.ini.
Chrome: ERR_CONNECTION_TIMED_OUT	A firewall drops the traffic: on the server, on the network, or (in a cloud) the security group.	On the server, Ubuntu: `sudo ufw status` (allow with `sudo ufw allow 8443/tcp`); Red Hat: `sudo firewall-cmd --list-ports` (allow with `sudo firewall-cmd --permanent --add-port=8443/tcp && sudo firewall-cmd --reload`). From the PC: `Test-NetConnection <server> -Port 8443` in PowerShell. Otherwise ask your network team to allow TCP 8443 from the examiners' network.
Chrome: ERR_SSL_PROTOCOL_ERROR, or the address starts with http://	Lucida on the network speaks HTTPS only.	Type **https://** in front of the address.
Service *activating (auto-restart)* or *failed*	Lucida stops right after starting.	`journalctl -u lucida -n 80` shows why. Common: the port is used by another program; the data disk is not mounted or full; workbench.ini was edited with a typo.
The models run on the CPU (`lucida gpu` says cpu)	The driver is missing or too old, the card is in use by something that took all its memory, or the CUDA device files are missing.	`nvidia-smi` must work (step 2) and show driver 525 or newer. Then `sudo nvidia-modprobe -u -c=0` and `sudo systemctl restart lucida`. `journalctl -u lucida | grep -i -E 'cuda|cudnn'` shows CUDA errors. Check that `ai_device` is not *cpu* in workbench.ini.
`nvidia-smi`: couldn't communicate with the NVIDIA driver	Driver not loaded: no reboot yet, Secure Boot rejected it, or nouveau is loaded.	Step 2.3.
A model is *not reproducible on the GPU*; it runs on the CPU	Two GPU runs gave different bits, so Lucida chose the CPU for that model to keep results reproducible.	Nothing to fix: the result is still correct. Report the driver version (`nvidia-smi`) with the self-test log.
Red Hat: the service fails with *Permission denied* although the files look right	SELinux refused something, usually a data folder outside /var/lib that was not labelled.	`sudo ausearch -m avc -ts recent` shows what was refused. Run `sudo restorecon -RF /opt/lucida /var/lib/lucida` (and your data folder), or `sudo ./install.sh` again, which does this.
The self-test FAILS on *installed files*	A program file was changed or removed after installation.	Run `sudo ./install.sh` again from the package; it replaces every file.
`lucida` asks for your password	It switches to the lucida account through sudo.	Type your own password. Your user must be allowed to use sudo (group *sudo* on Ubuntu, *wheel* on Red Hat).
Permission denied reading files to import	The lucida account cannot read your home folder.	Copy the files to `/var/lib/lucida/import` first.
Forgot a password		Another administrator resets it in **Users...** (it also ends that person's sessions). This is why step 8.2 asks for a second administrator.

What you see	Why	What to do
Lucida is slow with long videos; *range would not fit*	One frame range may use a sixth of RAM (about 10 GB on 64 GB) so the server never swaps.	Work on a shorter range, as the message suggests; or raise `range_mb` in workbench.ini if the server has nothing else to do.
Disk full	Evidence and derived video accumulate.	`df -h /var/lib/lucida`. Move the case data to a larger disk: stop Lucida, copy with `sudo rsync -aHAX /var/lib/lucida/ /srv/lucida/`, set `data_dir` and `import_dir`, run `sudo ./install.sh` again (it updates the service), check, then remove the old copy.

Collecting information for support

```
{ lucida version; lucida check; lucida gpu; nvidia-smi
  systemctl status lucida --no-pager
  journalctl -u lucida -n 300 --no-pager; } > ~/lucida-support.txt 2>&1
```

The file holds no evidence or passwords. Attach it, together with the newest log in `/var/lib/lucida/logs`, to your support request.

Quick reference

Task	Command
Check the server	`./check-server.sh (or: lucida check)`
Install or upgrade	`sudo ./install.sh`
Status / log	`systemctl status lucida / journalctl -u lucida -f`
Restart after a settings change	`sudo systemctl restart lucida`
Settings file	`/etc/lucida/workbench.ini`
Case data	`/var/lib/lucida`
CA certificate for examiner PCs	`/var/lib/lucida/tls/workbench-ca.crt (lucida ca)`
Self-test	`lucida selftest`
Where the models run	`lucida gpu`
Check installed files	`lucida verify-install`
Check evidence and audit log	`lucida verify`
Backup	`lucida backup --output FILE --encrypt`
Uninstall (keep data)	`sudo /opt/lucida/uninstall.sh`
Address for examiners	`https://<server address>:8443`

More: the user manual and the algorithm guide are in Lucida's Help page and in /opt/lucida/docs. Deployment details for every option: /opt/lucida/docs/deployment.md.